



실습 중심 강의

Coding Agent 기본

AI를 활용한 SW 개발 기초

PRD → GitHub → 배포 · 프롬프트 · 컨텍스트 · 하네스 엔지니어링을 직접 따라 하는 실습

실습 프로젝트

Tetris — 프롬프트 복붙으로 브라우저 게임을 만들어 GitHub Pages에 배포

개념부터 배포까지 — 프롬프트를 복붙해 브라우저 게임을 만들고 인터넷에 무료로 공개합니다.

과정 개요 —

항목	내용
교육 대상	프로그래밍 경험이 없어도 프롬프트 복붙으로 완주 — 누구나 참여
교육 시간	사전 준비 20-30분 + 이론(Part 0-6) → 실습(Part 7-8)
교육 방식	Codex App(데스크톱/웹)을 메인으로 '직접 손으로' 실습, CLI는 보조
도구 스택	순수 HTML/CSS/JavaScript · Node.js · Git · Codex App (설치할 것이 거의 없음)
사전 과제	계정: ChatGPT·OpenAI · GitHub / 설치: Node.js 18+ · Git
심화 옵션	게임 다듬기(선택) — 다음 블록 미리보기 · 레벨 · 속도 증가
관통 예제	Tetris — 프롬프트 복붙으로 만드는 브라우저 게임 (GitHub Pages 배포)

핵심 목표 — 프롬프트·컨텍스트·하네스로 PRD → GitHub → 배포까지, 내 손으로 제품을 출시·운영한다.

학습 목표 一

1

전체 기술 지도를 설명

Codex의 사용 방식·실행 환경·지시/권한 체계 등 전체 기술 지도를 설명할 수 있다.

2

규칙·검증 체계 구성

Prompt·Context·Harness와 AGENTS.md로 프로젝트 규칙과 검증 체계를 구성할 수 있다.

3

안전한 실행 설정

Sandbox·Approvals로 안전하게 실행하고 config.toml 기초를 설정할 수 있다.

4

제품 출시·운영

PRD → GitHub → 배포까지 제품을 출시·운영할 수 있다.



PART 1

시작 전 준비

계정과 환경

실습 중 '계정이 없어서' 막히지 않도록, 본격적인 실습 전에 계정과 도구를 먼저 준비합니다. 약 20-30분 정도 걸립니다.

🕒 1교시 · 이론

시작 전 준비 — 계정과 환경

공통

실습 중 막히지 않도록 계정·도구를 먼저 준비 (약 20-30분)

만들어 둘 계정 (모두 무료로 시작)

ChatGPT / OpenAI

Codex 로그인·모델 사용 — chatgpt.com

GitHub

소스 코드 저장·배포 트리거 — github.com

로컬 개발 도구 설치

필요한 것은 Node.js와 Git 뿐입니다.

설치 여부 확인

```
node -v
```

```
git --version
```

Node.js 18 이상 (20 LTS 권장)

사전 점검 체크리스트

✅ ChatGPT/OpenAI 로그인 가능

✅ GitHub 로그인됨

✅ node -v 버전 출력

✅ git --version 버전 출력



PART 2

Codex 개관

사용 방식(App·IDE·CLI)과 Codex App 설치

Codex가 무엇인지 전체 기술 지도를 개관하고, App·IDE·CLI 사용 방식과 Codex App 설치·첫 작업까지 살펴봅니다.

🕒 1교시 · 이론

Codex 전체 기술 지도 — 네 개의 축

공통

Codex는 터미널·에디터에서 실제로 코드를 읽고 고치고 실행하는 코딩 에이전트

1 모델

코드를 생성·추론하는 AI 엔진
(gpt-5.x-codex 계열)

다루는 곳 **Part 2**

2 사용 방식

Codex를 '어디서' 쓰는가 —
App · IDE 확장 · CLI

다루는 곳 **Part 2**

3 지시·권한

AGENTS.md(규칙) · Sandbox(
범위) · Approvals(승인)

다루는 곳 **Part 5**

4 하네스

빌드·테스트·배포·로그로 결과
를 '검증'하는 틀

다루는 곳 **Part 6**

이 네 축이 결과 품질을 좌우합니다: 무엇을 시키는가(프롬프트) · 무엇을 보여주는가(컨텍스트) · 어떤 검증 틀에서 일하게 하는가(하네스).

핵심 용어 미리 보기

공통

지금 다 외울 필요는 없습니다 — 각 용어는 해당 파트에서 실습으로 익힙니다

Prompt

Codex에게 시키는 자연어 지시. 작게 쪼개고 검증 방법을 함께 준다

Context

Codex가 참고하는 정보(파일·이미지·대화·AGENTS.md). 모델의 '작업 기억'

AGENTS.md

프로젝트 상시 규칙 문서. 모든 작업 전에 Codex가 먼저 읽는다

Harness

작업을 감싸는 검증·안전 장치 (빌드/테스트/린트/샌드박스)

Sandbox

Codex가 파일·네트워크를 쓸 수 있는 '기술적 범위'

Approvals

위험한 행동 전에 사람에게 물어보게 하는 '정책'

Thread

하나의 작업 세션. 여러 프롬프트가 이어진 대화

MCP

Codex에 외부 도구(GitHub 등)를 연결하는 표준 규약

사용 방식 — App · IDE · CLI

공통

어떤 방식으로 쓰든 실제로 일하는 AI 모델은 똑같습니다

App

MAIN

독립 데스크톱/웹 앱. 인앱 브라우저·변경
미리보기·승인 UI·클라우드 작업

언제 쓰나 **이 과정의 메인 실습**

IDE 확장

VS Code 등 에디터에 통합. 열린 파일·선택
영역을 자동 컨텍스트화

언제 쓰나 **코드 작성 중 즉시 도움**

CLI

터미널 기반. 스크립트/CI 자동화에 적합
(Rust로 작성되어 가볍고 빠름)

언제 쓰나 **자동화·보조**

[NOTE] 왜 App을 메인으로? App은 Codex가 제안한 파일 변경을 diff(전/후 비교)로 보여주고, 승인·미리보기·클라우드 작업을 한 화면에서 다룰 수 있어 입문자에게 가장 안전합니다.

▶ 실습 — App 설치부터 첫 작업까지

설치 → 폴더 열기 → 첫 지시 → 결과 확인

1 설치 & 로그인

openai.com에서 Codex 데스크톱 앱 설치 → 같은 ChatGPT 계정으로 로그인. 기본값은 '작업 폴더 안에서만 변경, 위험한 동작은 승인'

2 첫 프로젝트 열기

빈 폴더(예: codex-hello)를 만들고 App에서 '폴더 열기'. 이 폴더가 Codex의 작업 공간이 된다

3 첫 작업 — 인사하기

Codex App 작성 칸

폴더에 `hello.txt` 파일을 만들고
"Hello Codex" 한 줄을 써줘.

→ 파일 목록에서 `hello.txt` 를 열어
내용이 "Hello Codex"인지 확인

이어지는 실습(2-4): 다양한 일 시켜 보기

· 엑셀 VLOOKUP 예시 만들기

· 다운로드 폴더 정리 방법

· CSV 파일이 어떤 데이터인지 설명

· 이미지 5장 생성해 저장

Codex App 사이드바 핵심 기능

공통

웹 만들어 공유 · 반복 작업 자동화 · 기능 확장



사이트 (Sites)

프롬프트 한 줄로 웹페이지·앱·게임을 만들어 OpenAI 서버에 바로 호스팅. 미리보기
→ 공개 범위 선택 → 링크 공유

Codex에게 맡기는 배포



예약 (Scheduled)

'나중에·정기적으로' 할 일을 맡기기. 매일/매주 반복, 변화 감시. 실행 결과는 예약 화면에 수신함처럼 모임

반복 작업 자동화



플러그인 (Plugins)

새 기능(스킬·앱·템플릿)을 더하는 확장 장터. 문서·PDF·스프레드시트·프레젠테이션이 기본 포함

기능 확장

[TIP] 세 기능은 서로 연결됩니다 — 예약 작업이 플러그인·스킬을 함께 호출할 수 있습니다.



PART 3

프롬프트 엔지니어링

받은 만큼 일한다 — 작게 쪼개고, 검증을 함께

프롬프트는 Codex에게 주는 '작업 지시문'입니다. 핵심은 (1) 작업을 작고 명확하게 쪼개고, (2) '끝났는지 아는 법(검증)'을 함께 주는 것입니다.

🕒 1교시 · 이론

프롬프트 엔지니어링 — 나쁨 vs 좋음

공통

Codex는 받은 만큼 일합니다

× 나쁜 프롬프트

로그인 기능 만들어줘.

대상 파일 · 동작 규칙 · 검증 방법이 모두 빠짐
→ Codex가 알아서 추측 → 결과가 매번 달라진다

✓ 좋은 프롬프트

src/auth.ts 에 이메일+비밀번호 로그인을 구현하고,
실패 시 에러 반환. 완료 후 npm test로 검증해줘.

① 대상 파일 지정 ② 동작 규칙 ③ 반환 형태 ④ 검증 방법
→ 결과가 예측 가능하고, Codex가 스스로 검증까지

좋은 프롬프트의 두 가지 핵심 원칙

→ 이 둘을 매번 챙기는 체크리스트가 P·C·T·F

1 작업을 작고 명확하게 쪼갬다

큰 작업은 '먼저 계획을 세우라'고 시킨 뒤 한 단계씩 진행

2 '끝났는지 어떻게 아는지'를 함께 준다

테스트·실행·린트 등 스스로 검증할 방법을 포함

재사용 템플릿 — P · C · T · F

공통

네 글자만 외우면 거의 모든 작업에 적용되는 체크리스트 (순서 아님)



Persona / Role 역할·관점

어떤 전문가의 관점으로 일하게 할지 정한다

예) "보안을 중시하는 TypeScript 개발자로서"



Context 배경·제약

건드릴 파일 · 지켜야 할 규칙 · 참고 자료

예) @src/auth.ts 수정 · 비밀번호 평문 저장 금지



Task 할 일·검증

구체적으로 무엇을 하고, 성공을 어떻게 확인하는가

예) 로그인 구현 → 실패 시 에러 반환 → npm test 통과



Format 산출물 형태

결과를 어떤 형태로 받을지 정한다

예) 변경 요약과 diff로 보여줘

네 조각을 이어 붙이면 곧 12쪽의 '좋은 프롬프트' — 필요 없는 요소는 건너뛰어도 됩니다.

핵심 기법 — 결과를 바꾸는 6가지 도구

공통

P·C·T·F에 없으면 결과가 확 달라지는, 검증된 프롬프트 기법 (필요한 것만 골라 쓰기)

1

역할 부여 Role · Persona

누구의 관점으로 일하게 할지 정한다

예) "보안을 중시하는 TypeScript 개발자로서"

2

예시 제공 Few-shot

원하는 형식·패턴을 예시로 보여준다

예) 입력→출력 한 쌍을 붙여 출력 형식을 고정

3

계획 먼저 Plan · 단계적 사고

바로 고치지 말고 계획부터 세우게 한다

예) "수정 전에 먼저 계획을 제안해줘 (read-only)"

4

작게 쪼개기 To-Do

큰 일을 한 항목씩 체크리스트로 나눈다

예) "이 To-Do를 한 항목씩, 한 커밋씩 진행"

5

검증 지시 Verification

끝났는지 아는 법을 함께 준다

예) "구현 후 npm test 통과까지 확인"

6

반복 개선 Iterate

한 번에 X — 대화로 다듬는다

예) 안 되면 증상을 구체화하거나 새 스레드로

▶ 실습 — 나쁜 한 줄을 좋은 프롬프트로

공통

P·C·T·F와 핵심 기법을 세 가지 상황에 직접 적용해 봅니다

1

① 계획부터

공통

PRD로 To-Do 뽑기

✗ "테트리스 만들어줘"

✓ @PRD.md 를 읽고 — 수정하지 말고(read-only) — Tetris MVP 완성을 위한 To-Do를 체크리스트로 먼저 만들어 줘.

2

② 구현+검증

공통

한 기능을 검증까지

✗ "점수 기능 만들어줘"

✓ 꼼꼼한 프론트엔드 엔지니어로서 @script.js 에 줄이 삭제될 때 점수가 오르는 로직을 구현. 한 줄=100점, 여러 줄 동시 삭제 시 보너스. 완료 후 브라우저에서 점수가 정상적으로 오르는지 확인하는 방법을 알려줘.

3

③ 디버깅

공통

증상을 구체적으로

✗ "게임이 안 돼요, 고쳐줘"

✓ 블록이 좌우 벽을 통과해요. 벽 경계에서 멈추도록 충돌 검사를 고치고, 고친 뒤 브라우저에서 좌우 이동이 정상인지 확인하는 방법을 알려줘.

세 예시 모두 12·13쪽의 두 원칙과 P·C·T·F가 그대로 적용됩니다 — 대상·규칙·검증을 빠짐없이.



PART 4

컨텍스트 엔지니어링

네 가지 전략 — 작성·선택·압축·격리

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 알고 일하나'입니다. 필요한 정보를 '필요한 만큼만, 정확히' 보여 주는 기술입니다.

🕒 1교시 · 이론

컨텍스트 엔지니어링 — 네 가지 전략

공통

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 알고 일하나'

핵심 비유 — LLM은 컴퓨터, 컨텍스트 윈도우는 작업 기억(RAM). '많이 넣을수록 똑똑'은 오해 — 정확히, 관련 있는 것만.

● 작성 (Write)

중요한 규칙·결정을 밖에 적어 두기

→ AGENTS.md에 저장 (Part 5)

● 선택 (Select)

지금 작업에 필요한 것만 골라 넣기

→ @로 파일 코크 집어 주기

● 압축 (Compress)

길어진 대화를 요약해 자리 확보

→ 컴팩션 · 스레드 나누기

● 격리 (Isolate)

작업별로 맥락을 분리하기

→ 한 작업 = 한 스레드



PART 5

Codex 핵심 기술

지시·권한 체계 · AGENTS.md · config.toml · Sandbox/Approvals

AGENTS.md는 상시 규칙, config.toml은 기본 동작 설정, Sandbox·Approvals는 권한·승인을 정합니다. 이 셋이 '하네스'의 토대가 됩니다.

🕒 1교시 · 이론

AGENTS.md — 작업 전에 먼저 읽는 규칙서

공통

프로젝트의 상시 규칙(컨텍스트) · 계층적으로 합쳐짐

1

전역 (Global)

~/ .codex/AGENTS.md

내 모든 프로젝트 공통 개인 규칙

2

프로젝트

저장소 루트/AGENTS.md

이 프로젝트의 셋업·검증 규칙

3

하위 폴더

services/x/AGENTS.md

특정 폴더에만 다르게 적용



우선순위: 전역 → 프로젝트 루트 → 현재 폴더 (가까울수록 우선)

이 파일이 곧 '컨텍스트'이자 '하네스'

'변경 시 반드시 lint·test를 통과' 같은 규칙을 적어 두면, 매번 말하지 않아도 Codex가 스스로 지킵니다.

▶ 실습

- 전역 AGENTS.md 만들기
- 프로젝트 AGENTS.md 만들기
- 로드되는지 확인 요청하기

*config.toml — 모델·승인·샌드박스 기본값 저장
(~/ .codex/config.toml)*

Sandbox & Approvals — 안전하게 실행

공통

샌드박스 = '무엇을 할 수 있나(범위)' · 승인 = '언제 물어보나(시점)'

읽기 전용

```
sandbox = read-only
```

수정 없이 분석·계획만. 가장 안전

가장 안전

Auto (권장)

```
workspace-write + on-request
```

작업 폴더 내 자동, 폴더 밖·네트워크는 승인

균형 · 권장

Full access

```
danger-full-access + never
```

제한 없음. 격리된 VM/CI에서만

위험 · 격리 환경

네트워크는 기본 차단(보안). 패키지 설치처럼 필요할 때만 '이번에만 허용'을 누릅니다. App은 승인 모드 버튼 한 번으로 전환.



PART 6

하네스(Harness) 엔지니어링

빌드·테스트·배포·로그 검증

하네스는 Codex의 작업을 감싸는 '검증·안전 장치 묶음'입니다. 만든 결과가 '진짜 동작하는지' 자동으로 확인하게 만드는 틀입니다.

🕒 1교시 · 이론

Part 6. 하네스 엔지니어링 — 결과 검증의 틀

공통

Codex가 만든 결과가 진짜 동작하는지 자동으로 확인하게 만드는 안전망

1

빌드 (Build)

코드가 실제로 컴파일/번들되는
가

`npm run build / typecheck`

2

테스트 (Test)

기능이 의도대로 동작하는가

`npm test (Vitest)`

3

배포 (Deploy)

산출물을 실제로 배포할 수 있는
가

GitHub Pages

4

로그 (Log)

실행 결과·에러를 확인할 수 있는
가

테스트/런타임 로그

하네스 = '검증 가능한 규칙' + '권한 경계'. 일부러 코드를 깨보고(회귀 테스트), 빨간불이 났다가 수정 후 다시 통과하는 흐름을 직접 보면 '안전망'의 감이 옵니다.

여기까지가 공통 개념입니다

이제 실습을 시작합니다

앞에서 배운 원리(프롬프트·컨텍스트·하네스)를 그대로 Tetris 만들기에 적용합니다.

실습 프로젝트

Tetris — 브라우저 게임

Part 7-8 · 프롬프트 복붙 · GitHub Pages 배포

실습 프로젝트 한눈에

공통

프롬프트 복붙으로 만드는 브라우저 게임 — 이렇게 진행합니다

구분	공통
만드는 제품	Tetris · 브라우저 게임
코딩 사전지식	불필요 (프롬프트 복붙)
기술 스택	순수 HTML/CSS/JS
외부 연동	없음
배포 방식	GitHub Pages
모니터링	(선택) 없음
워크북 범위	Part 7-8

공통 트랙

테트리스 만들기 로드맵

Part 7 → 8 · 프로그래밍 경험 0에서도 프롬프트 복붙만으로 완주

7

PRD → To-Do

Tetris를 글로 정의

8.1-8.4

게임 만들기

보드·낙하·조작·점수를 프롬프트로

8.5

실행·테스트

시나리오 점검·디버깅

8.6

GitHub 올리기

커밋·푸시 (버튼으로도 가능)

8.7

GitHub Pages

누구나 접속하는 링크 배포

각 '프롬프트' 상자의 내용을 그대로 복사해 Codex 작성 칸에 붙여넣기만 하면 됩니다.

공통 · Part 7 — Tetris PRD → To-Do

공통 트랙

게임도 '무엇을 만들지' 글로 먼저 정의합니다

① PRD.md 작성

아래 내용을 그대로 붙여넣어 Tetris/PRD.md 로 저장하게 시킵니다.

```
# PRD — Tetris
```

```
## 목표
```

```
브라우저에서 도는 테트리스
```

```
## 핵심 기능
```

```
블록 낙하 · 회전 · 이동  
줄 삭제 · 점수 · 게임오버
```

② To-Do 도출 (읽기 전용)

@PRD.md 이 PRD를 읽고 Tetris MVP를 완성하기 위한 To-Do 목록을 만들어줘. 수정은 하지 마.

[TIP] 계획 단계는 read-only로 받으면 파일이 안 바뀌어 안전합니다.

PRD → To-Do 흐름은 어디서나 똑같습니다

PRD를 쓰고 → @로 참조해 To-Do를 받는 방법은 모든 프로젝트에 그대로 적용됩니다.

공통 · Part 8 — 프롬프트로 게임 만들기

공통 트랙

프롬프트 0 → 7 을 순서대로 복사·붙여넣기

0

프로젝트 골격

순수 HTML/CSS/JS로 빈 보드·점수 영역

2

자동 낙하 & 충돌

0.5초마다 한 칸 내려오고 바닥에서 고정

4

라인 클리어 & 점수

가득 찬 줄 삭제 → 위 블록 내리고 점수 증가

6

다듬기 (선택)

다음 블록 미리보기·레벨·속도 증가

1

보드 & 블록

그리드와 테트로미노(블록 모양) 정의·렌더링

3

키보드 조작

←→ 이동, ↓ 빨리, ↑ 회전, Space 즉시 낙하

5

게임 오버 & 재시작

새 블록 공간 없으면 종료·다시 시작

7

테스트 점검

시나리오 점검하고 안 되는 항목 고치기

[TIP] 문제가 생기면 증상을 구체적으로 적어 디버깅 프롬프트로 (예: "블록이 벽을 통과해요").

공통 · Part 8.6–8.7 — 배포하기

공통 트랙

만든 게임을 누구나 접속하는 링크로 무료 공개

1

Git으로 기록

코드가 동작하면 버전을 기록(commit). VS Code '소스 제어' 패널에서 버튼으로도 가능 — 세 단어만: add → commit → push

2

GitHub에 올리기

GitHub CLI(gh)로 인증·저장소 생성이 쉬움. 또는 웹에서 빈 저장소를 먼저 만들고 push

3

GitHub Pages 배포

저장소 Settings → Pages 에서 공개. 정적 파일(HTML/CSS/JS)이라 무료 배포 가능 → github.io 링크 생성

✅ 완료 조건 블록 낙하·조작·줄 삭제·점수·게임오버가 모두 동작하고, GitHub Pages 주소로 접속된다.

다시 함께 — 마무리 회고 & 부록

공통

이 자료로 개념을 정리하고 복습합니다

마무리 회고 (Part 13 / 공통)

- 전체 흐름 한눈에 보기
- 핵심 개념을 '한 문장으로' 말해 보기
- 최종 점검 체크리스트
- 자주 막히는 문제 트러블슈팅

부록 — 언제든지 꺼내 쓰는 참고

- A 프롬프트·컨텍스트·하네스 치트시트
- B config.toml 전체 예시(주석 포함)
- C AGENTS.md 템플릿(복사용)
- D 용어집

한 문장으로 말해 보기 — 개념 회고

프롬프트

작게 쪼개고 검증 방법을 함께 준다

컨텍스트

필요한 것만, 정확히 (많이 ≠ 똑똑)

AGENTS.md

작업 전 먼저 읽는 상시 규칙서

하네스

결과가 진짜 동작하는지 검증하는 안
전망

핵심 한 장 요약

이것만 기억하면 됩니다

Codex는 에이전트다

챗봇이 아니라 파일을 직접 고치고 테스트까지 돌린다

프롬프트

작게 쪼개고, 검증 방법을 함께 준다

컨텍스트

많이가 아니라 '정확히, 필요한 것만'

규칙 & 안전

AGENTS.md에 규칙, Sandbox/Approvals로 안전

하네스

빌드·테스트·배포·로그로 결과를 검증

실습은 직접

Tetris — 프롬프트 복붙으로 브라우저 게임을 만들어 배포



수고하셨습니다

개념은 함께, 실습은 각자 — 이제 내 손으로 만든 결과물이 인터넷에 있습니다.

다음 단계

- 부록 치트시트로 P.C.T.F를 내 프로젝트에 적용해 보기
- AGENTS.md에 나만의 '완료의 정의' 못 박기
- 게임에 다음 블록 미리보기 · 레벨 · 속도 증가 기능 더해 보기

Q & A · 질문을 환영합니다